

# One-Month Computer Science Internship Specification

## Project Title

**AI-Based Patient Risk Stratification and Scheduling Prioritization Using Python**

## Internship Duration

**Four weeks — approximately 160 hours**

## Student Eligibility

Third-year undergraduate student pursuing Computer Science, Data Science, Artificial Intelligence, Software Engineering, or a related discipline.

## Project Summary

The objective of this internship is to develop a Python-based prototype that reads and analyzes multiple healthcare data files and classifies patients according to their healthcare risk and scheduling priority.

The system will process the following types of patient data:

- Patient demographics
- Patient medications
- Patient laboratory results
- Patient care gaps
- Care-gap weight or priority assigned to each care gap

Using these inputs, the application will calculate a risk score for each patient and stratify patients into meaningful risk categories. It will also recommend whether a patient should be scheduled:

- Within the next one month
- Within the next two months
- Within the next three months

The solution should combine clearly defined business rules with an existing artificial intelligence or machine-learning model. The core application must be developed in Python.

The final output may be provided as a formatted Excel workbook, an interactive dashboard, or both.

# Business Problem

Healthcare organizations receive patient information from multiple data sources. Reviewing this information manually to determine which patients need immediate attention is time-consuming and inconsistent.

The proposed solution will consolidate patient information, calculate risk based on clinical and care-gap indicators, and create a prioritized scheduling list. This will help healthcare teams focus first on patients with the most urgent and highest-value care needs.

## Project Objectives

The intern will develop a working prototype that can:

1. Read multiple patient files in CSV or Excel format.
2. Validate the structure and quality of the source data.
3. Match records belonging to the same patient across files.
4. Consolidate the data into a patient-level analytical dataset.
5. Assign weightage to different care gaps.
6. evaluate demographic, medication, laboratory, and care-gap risk factors.
7. Calculate an overall risk score for each patient.
8. Stratify patients into high-, medium-, and lower-risk categories.
9. Recommend a scheduling timeframe of one, two, or three months.
10. Generate an Excel report or visualization showing the results.
11. Document the logic, assumptions, technical design, and instructions for running the application.

## Input Data

The prototype should support separate files containing the following information.

### 1. Patient Demographics

Suggested fields:

- Patient ID
- First name
- Last name
- Date of birth
- Age
- Gender
- ZIP code
- Primary care provider
- Clinic
- Last visit date
- Next scheduled appointment date
- Insurance or payer

## 2. Patient Medications

Suggested fields:

- Patient ID
- Medication name
- Medication category
- Prescription date
- Refill date
- Medication status
- Days supplied
- Medication adherence indicator, when available

## 3. Patient Laboratory Results

Suggested fields:

- Patient ID
- Laboratory test name
- Result value
- Units
- Reference range
- Abnormal-result indicator
- Test date
- Result status

Examples may include HbA1c, blood pressure, LDL cholesterol, kidney function, or other relevant clinical indicators.

## 4. Patient Care Gaps

Suggested fields:

- Patient ID
- Care-gap code
- Care-gap name
- Care-gap description
- Date care gap was identified
- Care-gap due date
- Care-gap status
- Last completed date
- Recommended frequency

## 5. Care-Gap Weightage

Suggested fields:

- Care-gap code
- Care-gap name
- Clinical priority
- Care-gap weight

- Revenue or quality impact, when applicable
- Maximum recommended scheduling timeframe

The files used for development must contain synthetic, de-identified, or otherwise approved data. Protected Health Information should not be stored in the student's personal development environment.

## Functional Requirements

### Data Ingestion

The application must:

- Read multiple CSV and Excel files.
- Allow file names and locations to be configured.
- Verify that required columns are present.
- Standardize column names and data types.
- Handle blank, duplicate, and invalid records.
- Produce a data-quality report showing rejected or incomplete records.

### Patient Record Integration

The application must:

- Join records using a unique patient identifier.
- Create one consolidated record per patient.
- Support patients having multiple medications, laboratory results, and care gaps.
- Use the most recent relevant laboratory result when multiple results are available.
- Count the number of open and overdue care gaps.
- Identify the highest-priority care gap for each patient.

### Feature Engineering

The prototype should create risk-related variables such as:

- Patient age
- Number of active medications
- Number of high-risk medications
- Number of abnormal laboratory results
- Severity of abnormal laboratory results
- Number of open care gaps
- Number of overdue care gaps
- Total care-gap weight
- Maximum individual care-gap weight
- Days since the patient's last visit
- Whether the patient already has an appointment
- Number of chronic-condition indicators, when available

### Risk-Scoring Logic

The patient risk score should combine multiple factors. A sample approach is:

**Overall Risk Score = Demographic Risk + Medication Risk + Laboratory Risk + Care-Gap Risk + Visit or Scheduling Risk**

Illustrative components may include:

- Age-based risk points
- Points for multiple active medications
- Points for selected high-risk medications
- Points for abnormal or critical laboratory values
- Sum of weighted open care gaps
- Additional points for overdue care gaps
- Additional points when the patient has not been seen recently
- Reduction in priority when an appropriate appointment is already scheduled

The exact formula must be configurable and documented rather than permanently hard-coded throughout the application.

## AI and Machine-Learning Component

The solution should use an existing machine-learning model or library rather than building a complex model from scratch.

Acceptable approaches include:

- Logistic regression
- Decision tree
- Random forest
- Gradient boosting
- K-means clustering for exploratory patient grouping
- A pretrained tabular classification model, where appropriate

Because labeled historical outcomes may not be available, the initial patient priority can be created using a rules-based score. The generated score or approved sample labels can then be used to test a machine-learning classifier.

The intern should compare at least two approaches:

1. Rules-based risk stratification
2. Machine-learning-based classification

The comparison should explain:

- Model inputs
- Model output
- Interpretability
- Accuracy or consistency
- Limitations
- Recommended approach for a future production implementation

Use of a generative AI model is optional. A generative AI model must not independently make clinical decisions or invent missing clinical information.

## Risk-Stratification Categories

The system should classify each patient into one of the following categories:

- **High Risk**
- **Medium Risk**
- **Lower Risk**

An optional fourth category may be used:

- **No Immediate Action or Already Scheduled**

Each classification must include an explanation of the primary factors that contributed to the assigned category.

Example:

High Risk — three overdue care gaps, elevated HbA1c result, high total care-gap weight, and no appointment scheduled.

## Scheduling-Priority Logic

The system must assign one of the following recommended scheduling windows.

### Schedule Within One Month

Possible criteria:

- High overall risk score
- Critical or significantly abnormal laboratory result
- Multiple overdue, high-weight care gaps
- High-priority care gap nearing or beyond its due date
- No existing appointment within the recommended timeframe

### Schedule Within Two Months

Possible criteria:

- Medium overall risk
- One or more moderately weighted care gaps
- Abnormal but noncritical laboratory result
- Patient requires follow-up but does not meet one-month criteria

### Schedule Within Three Months

Possible criteria:

- Lower current risk
- Preventive or routine care gaps
- Stable laboratory and medication indicators
- No immediate clinical urgency

## Already Scheduled or No Scheduling Needed

Use this classification when:

- An appropriate appointment already exists within the recommended period
- All applicable care gaps are closed
- Available information does not indicate an outstanding scheduling need

The recommendation must be a prioritization aid and not a replacement for clinical judgment.

## Expected Output

The final patient-level output should contain at least:

- Patient ID
- Patient name or synthetic identifier
- Age
- Clinic
- Primary care provider
- Number of active medications
- Number of abnormal laboratory results
- Number of open care gaps
- Number of overdue care gaps
- Total care-gap weight
- Highest-priority care gap
- Calculated risk score
- Risk category
- Recommended scheduling window
- Existing appointment date
- Primary reason for prioritization
- Data-quality or missing-data warning

## Excel Reporting Requirements

When Excel is selected as the output, the workbook should contain the following worksheets:

### Executive Summary

- Total number of patients analyzed
- Number and percentage in each risk category
- Number recommended for scheduling within one, two, and three months

- Number already scheduled
- Number of patients with missing or incomplete data

## Patient Priority List

A sortable patient-level table containing all risk and scheduling results.

## One-Month Scheduling List

Patients who should be contacted or scheduled within the next month.

## Two-Month Scheduling List

Patients who should be contacted or scheduled within the next two months.

## Three-Month Scheduling List

Patients who should be contacted or scheduled within the next three months.

## Care-Gap Summary

Summary by:

- Care-gap type
- Number of affected patients
- Average care-gap weight
- Number overdue
- Number in the one-month scheduling group

## Data-Quality Report

- Missing patient identifiers
- Duplicate records
- Invalid dates
- Missing laboratory values
- Unmatched patient records
- Missing care-gap weights

# Visualization Requirements

A dashboard may be developed using Streamlit, Power BI, Tableau, Plotly, or another approved tool.

Suggested visualizations include:

- Patients by risk category
- Patients by scheduling window
- Risk distribution by clinic



- Risk distribution by provider
- Most common open care gaps
- Highest-weight care gaps
- Patients with overdue care gaps
- Patients with abnormal laboratory results
- Patients who are high risk but have no scheduled appointment

The dashboard should support filtering by clinic, provider, payer, risk category, care gap, and scheduling window.

## Recommended Technology Stack

- Python 3.11 or later
- pandas
- NumPy
- scikit-learn
- openpyxl or XlsxWriter
- Matplotlib or Plotly
- Streamlit for an optional web dashboard
- Jupyter Notebook for exploratory analysis
- Git and GitHub for source control
- pytest for unit testing

## Proposed Python Application Structure

```

patient-risk-project/
|
├── data/
│   ├── input/
│   └── output/
|
├── config/
│   ├── care_gap_weights.csv
│   └── risk_rules.yaml
|
├── src/
│   ├── data_loader.py
│   ├── data_validator.py
│   ├── patient_matcher.py
│   ├── feature_engineering.py
│   ├── risk_engine.py
│   ├── scheduling_engine.py
│   ├── ml_model.py
│   ├── report_generator.py
│   └── main.py
|
└── tests/
    ├── test_data_loader.py
    ├── test_risk_engine.py
    └── test_scheduling_engine.py
  
```

```
|— notebooks/
|   |— exploratory_analysis.ipynb
|
|— requirements.txt
|— README.md
|— project_report.md
```

## Four-Week Internship Plan

### Week 1: Requirements, Data Analysis, and Design

The intern will:

- Understand the business problem and expected outcomes.
- Review all source-file layouts.
- Create a data dictionary.
- Identify required and optional fields.
- Develop Python routines to read and validate the files.
- Analyze missing values, duplicates, and invalid records.
- Design the consolidated patient data model.
- Document the proposed risk factors and scheduling rules.

#### Week 1 deliverables:

- Data dictionary
- Data-quality assessment
- Initial system design
- Working data-ingestion module
- Git repository with project structure

### Week 2: Data Integration and Rules-Based Risk Engine

The intern will:

- Combine patient information across files.
- Develop feature-engineering logic.
- Implement care-gap weight calculations.
- Develop the initial rules-based risk score.
- Assign high-, medium-, and lower-risk categories.
- Implement one-, two-, and three-month scheduling logic.
- Create unit tests for the principal rules.

#### Week 2 deliverables:

- Consolidated patient dataset
- Rules-based risk engine
- Scheduling-priority engine
- Preliminary patient-priority output
- Unit-test results

## Week 3: Machine Learning and Reporting

The intern will:

- Prepare the analytical dataset for machine learning.
- Train and test at least two appropriate models.
- Compare the models with the rules-based classification.
- Review feature importance or model coefficients.
- Generate the Excel workbook.
- Create initial charts or a Streamlit dashboard.

### Week 3 deliverables:

- Machine-learning notebook or module
- Model-comparison summary
- Initial Excel report
- Initial dashboard or visualizations

## Week 4: Testing, Refinement, and Presentation

The intern will:

- Test the prototype using multiple scenarios.
- Validate that the scheduling logic works as expected.
- Improve error handling and logging.
- Refine Excel reports and visualizations.
- Complete technical and user documentation.
- Prepare a final demonstration and presentation.
- Identify limitations and recommended next steps.

### Week 4 deliverables:

- Completed Python source code
- Final Excel report or dashboard
- Test cases and test results
- README and execution instructions
- Technical project report
- Final demonstration and presentation

## Required Deliverables

At the end of the internship, the student must submit:

1. Complete Python source code.
2. Requirements file listing all Python dependencies.
3. Sample synthetic input files.
4. Data dictionary.
5. Configurable care-gap weight file.
6. Configurable risk-rule file.
7. Consolidated patient-level output.

8. Excel workbook or interactive dashboard.
9. Unit tests and test results.
10. Model-comparison analysis.
11. README with setup and execution instructions.
12. Final technical report.
13. Ten- to fifteen-minute project demonstration.

## Acceptance Criteria

The prototype will be considered successfully completed when it can:

- Read all required source files without manual code changes.
- Detect missing or invalid required fields.
- Consolidate patient records accurately.
- Calculate care-gap weights and patient risk scores.
- Assign a risk-stratification category to every valid patient.
- Assign a one-, two-, or three-month scheduling recommendation.
- Identify patients who already have an appropriate appointment.
- Explain the main factors behind each patient's classification.
- Produce a readable Excel report or dashboard.
- Run through a documented Python command or application interface.
- Pass the agreed unit tests.

## Student Learning Outcomes

Upon completion, the student should gain experience in:

- Python application development
- Healthcare data processing
- Data cleaning and validation
- Data integration
- Feature engineering
- Rules-engine development
- Machine-learning classification
- Model interpretability
- Excel report automation
- Dashboard development
- Unit testing
- Git-based source control
- Technical documentation
- Responsible use of AI in healthcare